



Documentación Técnica

Introducción

TDBO es una aplicación web full stack diseñada para facilitar la organización económica de viajes en grupo.

La aplicación permite crear grupos de viaje, invitar miembros, registrar gastos con reparto equitativo o personalizado, consultar el historial de pagos y calcular balances individuales con sugerencias automáticas de liquidación.

La solución está pensada para reducir la fricción en la organización económica de viajes en grupo y centralizar toda la información relevante en una única plataforma.

El sistema se apoya en una arquitectura separada por capas, con un frontend desarrollado en React y Vite, un backend basado en Express y Sequelize, y una base de datos PostgreSQL.

Además, incorpora autenticación mediante JWT, control de acceso por membresía y documentación interactiva de la API mediante Swagger/OpenAPI.

Desarrolladores

Montse	https://github.com/Montse-gj
Marcos	https://github.com/marcossalinas26
Luis	https://github.com/lualvarimp
jonathan	https://github.com/r3dc0m

Arquitectura general

La aplicación sigue una arquitectura separada por capas, con un frontend independiente del backend y una API REST como punto de integración entre ambos. El frontend, construido con React, Vite y TypeScript, se encarga de la experiencia de usuario, la navegación y la gestión del estado de sesión; el backend, desarrollado con Express y TypeScript, concentra la lógica de negocio, la validación de acceso y la persistencia en base de datos.

Esta separación facilita el mantenimiento, la escalabilidad y la evolución del proyecto por módulos.

Tecnologías empleadas

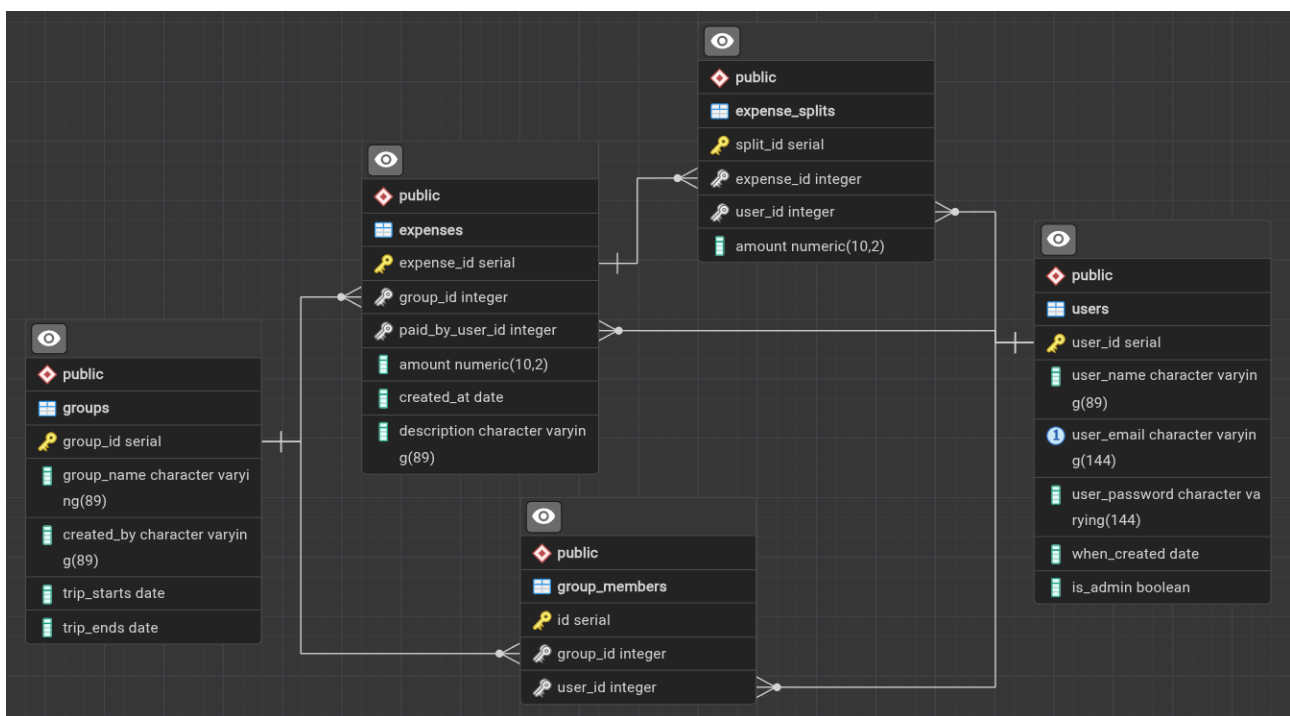
El frontend utiliza React Router para la navegación, React Context para la autenticación y hooks personalizados para encapsular la comunicación con la API.

El backend emplea Express como framework HTTP, Sequelize como ORM y PostgreSQL como base de datos relacional.

La autenticación se implementa con JWT, mientras que bcrypt se usa para proteger las contraseñas mediante hash antes de almacenarlas.

Además, la API se documenta con Swagger/OpenAPI y se expone a través de Swagger UI para facilitar pruebas y consumo.

Modelo de datos



El dominio funcional se organiza en torno a cinco entidades principales: usuarios, grupos, membresías de grupo, gastos y repartos de gasto.

- Un usuario puede crear grupos y pertenecer a varios viajes.
- Cada grupo tiene miembros asociados mediante una tabla intermedia.
- Cada gasto pertenece a un grupo y a un pagador.
- Cada gasto puede dividirse en varios splits vinculados a usuarios concretos.

Este modelo permite representar tanto gastos simples como repartos más complejos, manteniendo trazabilidad sobre quién pagó, quién participa y cuánto debe cada persona.

Backend y lógica de negocio

El backend se estructura en controladores especializados por funcionalidad: autenticación, usuarios, viajes y gastos.

auth.controller gestiona registro e inicio de sesión;

user.controller cubre perfil, búsqueda y listado;

trip.controller administra grupos, miembros e invitaciones;

expense.controller implementa la creación, actualización, eliminación y cálculo de balances.

La lógica de balances suma lo pagado por cada miembro, compara ese valor con lo que debe según los splits y genera transferencias sugeridas para simplificar la liquidación de cuentas.

Ese cálculo centralizado en servidor evita duplicidades y mantiene coherencia en los resultados.

Seguridad y control de acceso

El acceso a las rutas protegidas se controla mediante middleware JWT.

verifyToken valida el token, comprueba su firma y además verifica que el usuario siga existiendo en base de datos antes de dejar pasar la petición.

verifyGroupMembership añade una capa extra de protección, impidiendo el acceso a información de un grupo si el usuario no pertenece a él.

Esta combinación de autenticación y autorización por membresía es especialmente útil en una aplicación donde el aislamiento de datos entre viajes es esencial.

Frontend y flujo de uso

En el frontend, el contexto de autenticación persiste usuario y token en localStorage, de modo que la sesión se mantiene entre recargas.

Los hooks useTrips, useExpenses y useBalances encapsulan llamadas HTTP y simplifican el consumo de la API desde las páginas.

La navegación principal se organiza en vistas para inicio, login, registro, viajes, gastos, saldos y unión a un viaje por invitación.

La pantalla de invitación usa un flujo específico: si el usuario no está autenticado, se guarda el viaje pendiente y se redirige al login para continuar después con la unión.

API y documentación

La API está expuesta bajo el prefijo /api y agrupa rutas para autenticación, usuarios, viajes y gastos. El proyecto incorpora documentación con Swagger/OpenAPI, lo que permite describir rutas, cuerpos de petición, respuestas y esquemas reutilizables de forma estandarizada. Este enfoque es recomendable porque mejora la legibilidad, la mantenibilidad y el consumo de la API por parte de otros desarrolladores o herramientas de generación de clientes.

Despliegue y arranque

Todos los contenedores se lanzan con Docker Compose, tanto el frontend como el backend tienen sus Dockerfile respectivos para asignar volúmenes locales.

El backend arranca tras comprobar la conexión a la base de datos y sincronizar modelos con Sequelize; si no hay usuarios, ejecuta un seed inicial para cargar datos de ejemplo.

El frontend se sirve con Vite y redirige peticiones /api al backend mediante proxy en desarrollo.

Esta configuración simplifica el trabajo local, evita problemas de CORS durante la fase de desarrollo y acelera las pruebas de integración entre capas.

El proyecto se lanza con el siguiente comando:
`docker compose up -build`

Retos técnicos y decisiones de implementación

Retos técnicos

Al tratarse de un trabajo en grupo, una de las principales dificultades técnicas fue la comunicación y la distribución de tareas. Aunque en una fase inicial se determinaron responsabilidades por miembro del equipo con la ayuda de herramientas de gestión (como Trello), algunos esfuerzos individuales llegaron a solaparse debido a la finalización de funcionalidades en otros componentes, con o sin dependencia directa.

Uno de los problemas técnicos recurrentes estuvo relacionado con la integración de módulos de Node a medida que el proyecto avanzaba. A pesar de existir un intento de script en bash para reconstruir los contenedores Docker, no se logró una lectura clara de los errores de los contenedores en

relación con los módulos faltantes o problemas de sincronización de datos al cambiar los modelos.

También ocurrió que, en algunos casos, el módulo cors no estaba instalado en el momento en que se comenzaron a integrar las llamadas fetch al backend.

La URL utilizada por Vite no se ofuscó en el archivo `.env`, sino que quedó expuesta, asumiendo que el puerto utilizado por defecto no cambiaría. Esto generó cierta divergencia, ya que algunos desarrolladores tenían procesos activos de otros proyectos que utilizaban los mismos puertos, lo que provocaba que el servicio no se iniciara correctamente.

En otro caso, un usuario con sistema operativo basado en *NTFS* no podía utilizar el mapeo de volumen de la base de datos PostgreSQL debido a cómo el sistema interpreta las rutas relativas en comparación con sistemas tipo *FAT*. Para solventar este problema, se añadió al `.gitignore` un archivo que reemplaza selectivamente campos en el archivo `docker-compose.yml`, denominado `docker-compose.override.yml`.

La lógica de estilos que permite colapsar el panel lateral de detalles del grupo mediante un `useState` presentó cierta complejidad, ya que debía activar una gestión de clases capaz de mostrar u ocultar componentes en versión móvil. Debido a la presencia de múltiples estilos 'inline' y dos archivos CSS asociados a componentes específicos, fue necesario reorganizar los estilos a medida que avanzaba el proyecto, manteniendo simultáneamente la funcionalidad existente.

Decisiones de implementación

A mitad del proyecto, fue necesario incluir una nueva tabla intermedia para registrar desgloses por usuario, la cual no estaba contemplada inicialmente.

Por motivos de tiempo y para evitar solapamientos con el trabajo de estilos, aunque se consideró la implementación de un botón para eliminar usuarios de un grupo, se decidió no incluirlo debido a la complejidad de sus dependencias. Es decir, se debía optar entre permitir la eliminación solo si el grupo no tenía gastos registrados o implementar una confirmación previa en caso de que el usuario tuviera gastos pendientes, además de gestionar si dichos gastos debían reasignarse o mantenerse en la tabla intermedia. El diseño requerido excedía los recursos disponibles en el tiempo asignado.

Por último, otra funcionalidad que quedó como posible mejora futura —aunque podría haber formado parte del MVP— fue la opción de exportar el desglose de gastos a formatos PDF o CSV.

Conclusión

El proyecto ha sido muy interesante, no ha sido complicado proyectar una idea en sus inicios entre todos los miembros del equipo, y con la ayuda de las LLMs

se demostró que pueden llegar a finalizarse en plazos razonables proyectos simples y funcionales como TDBO.